

Aula 10 - Simulação de Atuadores e Modulação PWM

Descrição: Simulação virtual de controle de cargas, acionamento de relés, transistores, servomotores e saídas PWM.

Introdução

Na Aula 09, o nosso sistema ganhou “olhos, pele e ouvidos”: aprendemos a capturar luz, temperatura, presença e distância e transformar esses estímulos físicos em variáveis numéricas confiáveis dentro do código em C. Mas um sistema que apenas sente o ambiente é passivo — ele observa, mas não age.

Esta aula fecha o ciclo fundamental de todo sistema IoT: sentir → decidir → agir. Hoje vamos dar “músculos” ao projeto, aprendendo a converter os dados condicionados na Aula 09 em movimento e acionamento reais dentro do simulador: um servomotor que se move de acordo com a luminosidade, um relé que liga uma carga quando há presença detectada, e um motor DC cuja velocidade é modulada por PWM de acordo com a temperatura. O código de leitura da Aula 09 será literalmente reaproveitado como o “cérebro” de decisão desta aula.

Etapa 1 — O Universo dos Atuadores

Se um sensor é um transdutor que converte grandeza física em sinal elétrico, um atuador faz o caminho inverso: converte um sinal elétrico de controle em uma ação física (movimento, força, luz, calor). Na arquitetura do Arduino Uno, trabalharemos com três formas de atuação:

1. **Atuadores On/Off via relé:** um módulo relé funciona como uma chave eletromecânica controlada por um sinal digital (HIGH/LOW) do Arduino. Ele isola eletricamente o circuito de baixa tensão do microcontrolador do circuito de carga (que pode ser de tensão/corrente muito mais altas), por meio de uma bobina que aciona um contato mecânico.
2. **Atuadores contínuos via PWM — Servomotor:** controlado pela biblioteca Servo.h, o servo interpreta a largura de um pulso elétrico (tipicamente entre 1 ms e 2 ms) para posicionar seu eixo entre 0° e 180°. O comando `servo.write(angulo)` abstrai esse cálculo de pulso para o programador.
3. **Atuadores contínuos via PWM — Motor DC controlado por transistor:** o Arduino não tem corrente suficiente em seus pinos para acionar diretamente um motor. Por isso, usamos um transistor como uma “chave eletrônica”: um pequeno sinal na base (vindo do Arduino, sempre por meio de um resistor limitador) controla uma corrente muito maior entre coletor e emissor, que alimenta o motor.

PWM (Pulse Width Modulation) é a técnica central desta aula: em vez de uma saída puramente HIGH ou LOW, o Arduino simula um valor “analógico” de saída variando a proporção de tempo em nível alto dentro de cada ciclo (o duty cycle). A instrução `analogWrite(pino, valor)` aceita valores de 0 (0% do tempo em HIGH, desligado) a 255 (100% do tempo em HIGH, potência máxima), e só funciona nos pinos digitais marcados com til (~), no Uno: 3, 5, 6, 9, 10 e 11.

Atenção — revisão obrigatória da montagem da Aula 09 (resistores)

No material da Aula 09, a Etapa 2 explicou corretamente que o LDR precisa de um divisor de tensão com um resistor de 10 kΩ para gerar uma leitura confiável no pino A0. Porém essa peça não foi incluída na lista de componentes da Prática 1 nem citada no passo a passo de “Montagem base” — uma omissão que pode ter deixado o resistor de fora do circuito de alguns grupos, mesmo com o código funcionando normalmente no simulador (o valor lido só fica incorreto/instável, o que é fácil de passar despercebido). Antes de iniciar a Aula 10, confira no projeto Tinkercad da Aula 09:

- O LDR está associado a um resistor de 10 kΩ formando um divisor de tensão (LDR entre 5V e o pino A0; resistor de 10 kΩ entre A0 e GND).
- Sem esse resistor, o pino A0 fica “flutuando” eletricamente e os valores de `analogRead()` deixam de refletir a luminosidade real.

Esse cuidado é ainda mais importante hoje: vamos introduzir um transistor no circuito, e todo pino de controle de um transistor a partir do Arduino também exige um resistor limitador de corrente (resistor de base). As listas de materiais e os passos de montagem desta aula detalham cada resistor necessário — verifiquem item a item antes de simular.

Prática 1 — Montagem e teste isolado dos atuadores

Reabram o mesmo projeto Tinkercad da Aula 09 (**não criem um projeto novo** — vamos reaproveitar o circuito de sensores). Adicionem:

- 1 Servomotor (Micro Servo)
- 1 Módulo Relé de 1 canal
- 1 Transistor NPN (ex.: 2N2222)
- 1 Motor DC pequeno
- 1 Diodo 1N4007 (proteção contra pico de tensão reversa do motor)
- 1 Resistor de 1 kΩ (resistor de base do transistor — obrigatório)
- Confirmem que o resistor de 10 kΩ do LDR (Aula 09) continua presente

Montagem base:

- **Servomotor:** fio de sinal (laranja/amarelo) no pino digital 9 (PWM); alimentação no barramento 5V/GND.
- **Módulo relé:** pino IN no pino digital 4; VCC/GND no barramento.
- **Transistor + motor DC:** base do transistor ligada ao pino digital 10 (PWM) através do resistor de 1 kΩ; coletor no terminal negativo do motor; emissor no GND; terminal positivo do motor no 5V; diodo 1N4007 em paralelo com o motor, com a faixa (catodo) voltada para o polo positivo, permitindo o retorno seguro da corrente induzida quando o motor é desligado.

Bloco de Código 1 — Movimento bruto dos atuadores

```
#include <Servo.h>
```

```
const int PINO_SERVO = 9;    // Saida PWM - servomotor
```

```
const int PINO_RELE = 4;     // Saida digital - rele
```

```
const int PINO_MOTOR_PWM = 10; // Saida PWM - base do transistor (via resistor 1k)
```

```
Servo meuServo;

void setup() {
  Serial.begin(9600);
  meuServo.attach(PINO_SERVO);
  pinMode(PINO_RELE, OUTPUT);
  pinMode(PINO_MOTOR_PWM, OUTPUT);
}

// Voce deve rodar este laço e observar o movimento dos atuadores no simulador
void loop() {
  // Teste do servo: varre de 0 a 180 graus
  for (int angulo = 0; angulo <= 180; angulo += 30) {
    meuServo.write(angulo);
    Serial.print("Servo: "); Serial.println(angulo);
    delay(500);
  }

  // Teste do rele: liga e desliga
  digitalWrite(PINO_RELE, HIGH);
  Serial.println("Rele: LIGADO");
  delay(1000);
  digitalWrite(PINO_RELE, LOW);
  Serial.println("Rele: DESLIGADO");
  delay(1000);

  // Teste do motor DC via PWM (rampa de velocidade)
  for (int pwm = 0; pwm <= 255; pwm += 51) {
    analogWrite(PINO_MOTOR_PWM, pwm);
    Serial.print("Motor PWM: "); Serial.println(pwm);
    delay(500);
  }
  analogWrite(PINO_MOTOR_PWM, 0);
}
```

Explicação do código:

Este código foca no isolamento de hardware para testar individualmente cada componente e garantir que não haja erros de montagem elétrica:

- **Inclusão de Biblioteca e Mapeamento:** O comando `#include <Servo.h>` carrega a biblioteca de controle de servomotores. Definem-se constantes para os pinos: pinos PWM 9 e 10 para o servomotor e a base do transistor do motor DC, e o pino digital 4 para a bobina do relé.

- **Configuração no `setup()`:** Instancia o objeto `meuServo` e associa seu pino via `.attach(9)`, o que delega ao temporizador interno do Arduino a geração automática dos pulsos PCM. O Monitor Serial é iniciado a 9600 bps e os pinos do relé e do motor são declarados como saídas com `pinMode(..., OUTPUT)`.
- **Varredura Angular do Servomotor:** O laço `for` incrementa o ângulo de 0° a 180° em passos de 30°. O método `meuServo.write()` converte a variável numérica em pulsos de largura específica (1 ms a 2 ms), reposicionando o eixo mecânico do servo a cada meio segundo.
- **Acionamento Binário do Relé:** Aciona a chave eletromecânica do relé em nível alto (**HIGH**) e baixo (**LOW**) com `digitalWrite()`, permitindo validar visual e audivelmente o chaveamento da carga isolada.
- **Rampa de Velocidade por PWM no Motor DC:** Um laço `for` eleva o *Duty Cycle* do pino 10 de 0 a 255 em incrementos de 51 unidades. A instrução `analogWrite()` altera a proporção de tempo em que o pino permanece em nível alto, variando a corrente na base do transistor e criando uma rampa de aceleração de 5 níveis no motor DC antes de zerar a saída.

Etapa 2 — Condicionadores de Atuação: do dado à ação

Assim como na Aula 09 precisamos condicionar o dado bruto do sensor, aqui precisamos condicionar a decisão: transformar um valor já tratado (percentual de luz, graus Celsius, estado do PIR) em um comando de atuação dentro da faixa que cada atuador aceita.

- **`map()`:** reescala um valor de uma faixa de entrada para uma faixa de saída. Usaremos `map(luzPercentual, 0, 100, 0, 180)` para converter o percentual de luminosidade (0–100) no ângulo do servo (0–180°).
- **`constrain()`:** garante que um valor nunca ultrapasse os limites seguros de um atuador (por exemplo, manter o PWM sempre entre 0 e 255), mesmo que o cálculo intermediário resulte em um número fora da faixa.
- **Motor DC / PWM por temperatura:** definimos um limiar de segurança (`LIMIAR_TEMP`); abaixo dele o motor permanece desligado (`PWM = 0`); acima dele, a velocidade cresce proporcionalmente à temperatura, sempre restrita pelo `constrain()`.
- **Relé por presença:** lógica binária simples a partir do PIR — HIGH liga, LOW desliga. Atenção: alguns módulos de relé reais são “ativos em LOW” (o relé liga quando o pino recebe 0V); no simulador, verifiquem o comportamento do componente antes de fixar a lógica.

Prática 2 — Implementando as funções de atuação

Montagem: a mesma da Prática 1 — sem alterações físicas. Reforcem que o resistor de base do transistor (1 kΩ) está de fato entre o pino 10 e a base do componente.

Bloco de Código 2 — Condicionamento lógico de atuação

```
#include <Servo.h>

const int PINO_SERVO = 9;
const int PINO_RELE = 4;
const int PINO_MOTOR_PWM = 10;
const float LIMIAR_TEMP = 28.0; // graus Celsius - a partir daqui o motor liga
```

```
Servo meuServo;
```

```
// Converte o percentual de luz (0-100) em angulo do servo (0-180)
```

```
void moverServoPorLuz(int luzPercentual) {  
  int angulo = map(luzPercentual, 0, 100, 0, 180);  
  meuServo.write(angulo);  
}
```

```
// Converte temperatura em duty cycle do motor DC, respeitando o limiar
```

```
void controlarMotorPorTemperatura(float tempCelsius) {  
  int pwm = 0;  
  if (tempCelsius > LIMIAR_TEMP) {  
    pwm = map((int)tempCelsius, (int)LIMIAR_TEMP, 50, 80, 255);  
    pwm = constrain(pwm, 0, 255); // nunca deixa o PWM sair de 0-255  
  }  
  analogWrite(PINO_MOTOR_PWM, pwm);  
}
```

```
// Liga ou desliga o rele conforme o estado do PIR
```

```
void controlarRelePorPresenca(int presenca) {  
  digitalWrite(PINO_RELE, presenca == HIGH ? HIGH : LOW);  
}
```

```
void setup() {  
  Serial.begin(9600);  
  meuServo.attach(PINO_SERVO);  
  pinMode(PINO_RELE, OUTPUT);  
  pinMode(PINO_MOTOR_PWM, OUTPUT);  
}
```

```
void loop() {  
  // Dados ficticios de teste (validando a logica antes de integrar com a Aula 09)
```

```
  int luzSimulada = 70;  
  float tempSimulada = 32.5;  
  int presencaSimulada = HIGH;
```

```
  moverServoPorLuz(luzSimulada);  
  controlarMotorPorTemperatura(tempSimulada);  
  controlarRelePorPresenca(presencaSimulada);
```

```
  Serial.print("Servo: "); Serial.print(map(luzSimulada, 0, 100, 0, 180)); Serial.println(" graus");  
  Serial.print("Motor: "); Serial.println(tempSimulada > LIMIAR_TEMP ? "ativo" : "desligado");
```

```
Serial.print("Rele: "); Serial.println(presencaSimulada == HIGH ? "LIGADO" : "DESLIGADO");  
Serial.println("-----");  
  
delay(1500);  
}
```

Explicação do código:

Este bloco abstrai os atuadores em funções dedicadas, separando a lógica matemática de conversão das leituras do circuito físico:

- **Mapeamento Linear do Servo (`moverServoPorLuz`):** A função recebe a luminosidade tratada em percentual (0 a 100%) e aplica `map(luzPercentual, 0, 100, 0, 180)`. Essa equação converte proporcionalmente o valor percentual para a escala angular em graus exigida pelo método `.write()`.
- **Controle Térmico Limiarizado (`controlarMotorPorTemperatura`):** A função compara a temperatura recebida com a constante `LIMIAR_TEMP` (28.0 °C). Se o valor for superior, o intervalo de temperatura (entre o limiar e 50 °C) é reescalado via `map()` para um *Duty Cycle* de 80 a 255. O uso de `constrain(pwm, 0, 255)` atua como uma trava de segurança que impede que extrapolações matemáticas gerem valores fora do limite de 8 bits do PWM.
- **Acionamento por Operador Ternário (`controlarRelePorPresenca`):** Utiliza a estrutura condicional `presenca == HIGH ? HIGH : LOW` no `digitalWrite()` para garantir a escrita do estado digital direto do sensor de presença no pino do relé.
- **Validação por Dados Fictícios no `loop()`:** Antes de integrar os sensores reais, o `loop()` utiliza variáveis estáticas (`luzSimulada = 70`, `tempSimulada = 32.5`, `presencaSimulada = HIGH`). Essa prática de engenharia de software isola bugs de código da leitura errática de hardware, confirmando o processamento lógico das funções através da impressão organizada no Monitor Serial.

Checklist de resistores para o circuito final da Aula 10

- Resistor de 10 kΩ no divisor de tensão do LDR (revisão da Aula 09 — confirmar antes de simular).
- Resistor de 1 kΩ na base do transistor que aciona o motor DC (limita a corrente vinda do pino PWM do Arduino; sem ele, o pino ou o transistor podem ser danificados).
- Diodo 1N4007 em paralelo com o motor DC, protegendo o transistor contra o pico de tensão reversa ("flyback") gerado pela bobina do motor ao desligar.
- Servomotor e módulo relé são alimentados diretamente pelo barramento 5V/GND, sem resistor adicional — o módulo relé já traz internamente seu próprio transistor, diodo e resistor de acionamento.

Etapa 3 — O Desafio do Protagonismo

Chegou o momento de unificar os dois cérebros do projeto: o código de leitura e condicionamento de sinais da Aula 09 e as funções de atuação construídas hoje. O resultado é um único firmware que fecha o ciclo sentir → decidir → agir.

1. Dinâmica da atividade

- Copiem para um novo sketch (ou reaproveitem o mesmo) todas as constantes de pino e as funções `condicionarTemperatura`, `condicionarLuminosidade` e `lerDistanciaUltrassonica` da Aula 09, sem alterações.
- Adicionem as constantes de pino e as funções `moverServoPorLuz`, `controlarMotorPorTemperatura` e `controlarRelePorPresenca` desta aula.
- No `loop()`, leiam e condicionem os sensores (como na Aula 09) e, na sequência, chamem as três funções de atuação passando os valores já condicionados.
- Utilizem estrutura condicional (`if/else`) para compor a mensagem de status de segurança a partir do PIR, exatamente como na Aula 09.

2. Especificações técnicas requeridas

- As funções de condicionamento da Aula 09 devem ser reaproveitadas integralmente — não recalculam luminosidade, temperatura ou distância com lógica nova.
- O uso de `map()` (conversão de faixas) e `constrain()` (proteção de limites) é obrigatório nas funções de atuação.
- O motor DC só pode ser acionado (`PWM > 0`) quando a temperatura ultrapassar o `LIMIAR_TEMP` definido; abaixo disso, o PWM deve ser 0.
- O relé deve refletir o estado do PIR a cada ciclo do `loop()`, em tempo real.
- O Monitor Serial deve imprimir, a cada ciclo, uma única linha de telemetria reunindo os dados dos sensores E o estado dos atuadores.
- Mantenha o `delay()` de estabilização ao final do `loop()`.

3. O entregável

Submetam o link de compartilhamento do projeto Tinkercad unificado (sensores da Aula 09 + atuadores da Aula 10). A entrega é composta por três frentes de avaliação:

- **Circuito físico virtual:** montagem completa dos sensores (Aula 09) e atuadores (Aula 10) na protoboard, com todos os resistores presentes — ver quadro de atenção a seguir.
- **Código-fonte em C polido:** indentação correta, funções modulares reaproveitadas e novas, e comentários pontuais explicando a lógica de conversão sensor → atuador.
- **Console de saída formatado:** linha de telemetria única por ciclo, no padrão do exemplo abaixo:

Temp: 32.50 C | Luz: 70% | Dist: 45 cm | Servo: 126 graus | Motor PWM: 180 | Rele: LIGADO | Status:
ALERTA: Presenca Detectada

(Saída gerada quando o PIR estiver HIGH e a temperatura acima do limiar)

Temp: 24.10 C | Luz: 40% | Dist: 118 cm | Servo: 72 graus | Motor PWM: 0 | Rele: DESLIGADO | Status:
Area Segura

(Saída gerada quando o PIR estiver LOW e a temperatura abaixo do limiar)